Brickts SQL course — Week 10 — Operate PostgreSQL and Timescale

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 10 — Operate PostgreSQL and Timescale

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 64: Day 64 — Create a time-partitioned scratch hypertable
- Day 65: Day 65 — Measure chunk pruning and time buckets
- Day 66: Day 66 — Track continuous aggregate freshness
- Day 67: Day 67 — Review retention and columnstore as data lifecycle changes
- Day 68: Day 68 — Budget database memory and avoid full-history fetches

- Day 69: Day 69 — Measure bounded scale with declared limits
- Day 70 (weekly challenge): Day 70 — Independent challenge — a Timescale performance report

---

Day 64 — Create a time-partitioned scratch hypertable

Previous: Day 63 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–63, or equivalent SQL skills.

Goal

Explain what a Timescale hypertable changes and preserve sample identity when partitioning by time.

Before you start

Use the Timescale lab described in Lab guide. Week 10 requires the extension: a plain PostgreSQL session cannot execute these examples. Record `version()` and `pg_extension.extversion`; the validated baseline is PostgreSQL 16.15 with TimescaleDB 2.30.2. If unavailable, record the blocked lab rather than reporting a pass. Never install extensions on a shared server.

Concept

A hypertable presents one SQL table while storing rows in time chunks. Partitioning supports pruning and chunk lifecycle operations; it does not remove the need for selective predicates or indexes. Every unique constraint must include all partition dimensions. With time as the only dimension, `(site_id, point_id, ts)` remains valid.

A unique `(site_id, point_id)` would incorrectly forbid history and cannot enforce uniqueness across time partitions. See Timescale's unique-index requirements.

Tiny example

```
SELECT extversion FROM pg_catalog.pg_extension
WHERE extname = 'timescaledb';

CREATE TABLE student.chunk_demo (
  ts timestamptz NOT NULL, item integer NOT NULL,
  PRIMARY KEY (item, ts)
);
SELECT create_hypertable(
  'student.chunk_demo', by_range('ts', INTERVAL '12
        hours'));
```

Use an empty scratch name. This supported conversion API makes the time dimension explicit without creating a wall-clock lifecycle policy.

SQL challenge

Create `student.ts_sample` with the same six columns, quality/finite checks and logical key as `course.sample`. Convert the empty table to a hypertable with twelve-hour chunks, then copy exactly the fixture rows. Keep metadata in `course`; no extra hash dimension is needed. Inspect `timescaledb_information.hypertables`, `dimensions` and `chunks`. State whether this scratch copy includes foreign keys, and why that matters.

Experiment

In a separate disposable table, attempt a unique key that omits `ts`; record the rejection and recover cleanly. Inspect the indexes actually created on the valid hypertable instead of assuming a particular name.

Acceptance checks

- The copy contains 24,190 rows and preserves the one bad/null value.
- Four twelve-hour chunks cover the two-day fixture.
- Repeating a logical key is rejected.
- The extension version and partition/index definitions are saved.

Stretch and reflection

How would adding a second partition dimension change every unique constraint? What workload would justify it?

Save your SQL, actual output, and one explanation in `student-work/sql/day64/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 63 ·

Day 65 — Measure chunk pruning and time buckets

Previous: Day 64 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–64, or equivalent SQL skills.

Goal

Use explicit time bounds to reduce chunk work and compare Timescale buckets with PostgreSQL's fixed-duration bins.

Before you start

Use `student.ts_sample` from day64. Keep the source fixture unchanged. Record `TimeZone`; use UTC literals and an explicit origin for comparisons. Inspect plans with a local statement deadline.

Concept

A bucket is a grouping key, not a missing-data repair. `time_bucket` offers Timescale functionality, while PostgreSQL `date_bin` already supports fixed positive intervals. Neither invents an observation for an empty interval. Match origins before comparing results. A filter on the raw timestamp makes the intended range clear and can allow chunk exclusion. Wrapping the timestamp in a date expression can change optimization and timezone meaning; judge the actual plan.

Tiny example

```
SELECT
  date_bin(INTERVAL '15 minutes', ts,
           '2026-01-05 00:00Z'::timestamptz) AS
         pg_bin,
  time_bucket(INTERVAL '15 minutes', ts,
              '2026-01-05 00:00Z'::timestamptz) AS
         ts_bin,
  count(*) AS source_rows
FROM student.ts_sample
WHERE site_id = 'SYN-NORTH'
  AND ts >= '2026-01-05 08:00Z'::timestamptz
  AND ts < '2026-01-05 09:00Z'::timestamptz
GROUP BY pg_bin, ts_bin ORDER BY pg_bin;
```

Four equal bucket pairs appear. Their combined source count is 334: two AHUs times fourteen roles times twelve instants, minus two absent samples. Mixed roles are counted here, never averaged together.

SQL challenge

For north Supply A duct static only, return four fifteen-minute buckets for that hour with average Pa, eligible count and expected count three. The counts should be 3, 2, 2, 3. Compare both bucket functions. Capture bounded plans for this one-hour interval and for both days; annotate chunks scanned or excluded using the nodes your installed version actually reports.

Experiment

Compare raw timestamp predicates with a `ts::date` predicate for January 5. Preserve identical intended UTC semantics, then explain any plan difference. Do not force a plan merely to obtain a desired screenshot.

Acceptance checks

- Only one resolved role/unit is averaged.
- Bucket origins and half-open bounds agree.
- Coverage exposes the missing samples without interpolation.
- Plan evidence identifies the time restriction and chunk work.

Stretch and reflection

Why can calendar-month or daylight-saving buckets require a different comparison from a fixed fifteen-minute interval?

Save your SQL, actual output, and one explanation in `student-work/sql/day65/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 64 ·

Day 66 — Track continuous aggregate freshness

Previous: Day 65 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–65, or equivalent SQL skills.

Goal

Separate a continuous aggregate's stored answer from the freshness of its source data.

Before you start

Use the scratch hypertable from day64. Keep policy scheduling off for these historical January fixtures. Work with explicit refresh intervals and record each source edit and refresh. Run refresh calls in autocommit, outside `BEGIN`/`COMMIT`; commit source corrections first.

Concept

A continuous aggregate stores summaries maintained by refresh work. Its materialized data can lag behind inserts or corrections. Real-time behavior is a separate setting, and does not automatically repair every historical change. A response therefore needs a source range, refresh information and a freshness policy. Timescale's refresh guide explains the distinction between refresh offsets and their execution schedule. A current-time policy may never revisit this old fixture.

Tiny example

```
CREATE MATERIALIZED VIEW student.hourly_value
WITH (timescaledb.continuous,
      timescaledb.materialized_only = true) AS
SELECT site_id, point_id, time_bucket(INTERVAL '1
         hour', ts) AS bucket,
        avg(value) AS mean_value, count(*) AS
          good_count
FROM student.ts_sample
WHERE quality = 'good' AND value IS NOT NULL
GROUP BY site_id, point_id, bucket
WITH NO DATA;

CALL refresh_continuous_aggregate('student.hourly_value',
  '2026-01-05 00:00Z'::timestamptz,
  '2026-01-07 00:00Z'::timestamptz);
```

Each row has one point's hourly mean in that point's unit. Mixed units remain separate by UUID.

SQL challenge

Compare the materialized and raw hourly answers for north Supply A kW at January 5, 15:00 UTC. Both should have eleven eligible rows averaging 12 kW. Change one eligible scratch source value from 12 to 24, commit, and capture the stale materialized answer. Refresh exactly that hour; the new mean should be `144.0/11`. Restore the source value and refresh again.

Experiment

Add a historical correction outside a hypothetical policy's refresh window. Explain why running that policy would not necessarily fix it. Record a proposed bounded backfill procedure; do not create an open-ended refresh job.

Acceptance checks

- Before, stale and refreshed results are all preserved.
- The eleven-row denominator is explicit.
- Source restoration returns the original answer.
- The design distinguishes source revision, completed refresh and response time.

Stretch and reflection

When could retaining old aggregates while deleting raw data conflict with a later refresh?

Save your SQL, actual output, and one explanation in `student-work/sql/day66/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 65 ·

Day 67 — Review retention and columnstore as data lifecycle changes

Previous: Day 66 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–66, or equivalent SQL skills.

Goal

Measure a small columnstore conversion and demonstrate retention with an absolute, reviewable cutoff.

Before you start

Make disposable scratch copies; `course.sample` must remain intact. Keep day66's aggregate source available. January fixtures are already old relative to today, so do not attach a relative-time retention job to them.

Concept

Retention drops whole eligible chunks, changing which questions remain answerable. Columnstore changes storage and query tradeoffs while keeping rows queryable. Compare checksums or ordered values before and after conversion. The 2.30.2 policy source defines `add_columnstore_policy` as a procedure; use `CALL`, while `add_retention_policy` is a function. The matching maintenance source defines `convert_to_columnstore` as a procedure too. Record installed signatures when using another release; old compression examples are not the course API.

Tiny example

```
SELECT n.nspname, p.proname, p.prokind,
       pg_catalog.pg_get_function_arguments(p.oid)
         AS arguments
FROM pg_catalog.pg_proc AS p
JOIN pg_catalog.pg_namespace AS n ON n.oid =
         p.pronamespace
WHERE p.proname IN ('add_columnstore_policy',
                    'convert_to_columnstore',
         'add_retention_policy')
ORDER BY n.nspname, p.proname;
```

`prokind = 'p'` identifies a procedure and `f` a function. This is evidence about the running installation.

SQL challenge

Create a separate twelve-hour-chunk hypertable copy. Enable columnstore using `timescaledb.enable_columnstore`, segment by site/point and order by time according to the installed release. Convert one explicitly identified scratch chunk with `CALL convert_to_columnstore(...)`. Compare one bounded history query's rows and plan before and after. Record size information where supported; tiny fixtures need not save space.

Experiment

On another disposable copy, inspect which chunks precede the absolute cutoff January 6 at 00:00 UTC. Use `drop_chunks` with that timestamp, then verify only January 6 remains: 12,096 rows. Capture eligibility before deleting anything. Explain how a refresh that revisits deleted raw history could change retained aggregates.

Acceptance checks

- Conversion preserves the selected row set and quality.
- Retention affects only the named scratch table and declared chunks.
- Source fixtures and day66's restored answer remain unchanged.
- No background lifecycle job is left running.

Stretch and reflection

Which legal or operational requirement would determine retention in a real deployment? Record it as a requirement to investigate.

Save your SQL, actual output, and one explanation in `student-work/sql/day67/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 66 ·

Day 68 — Budget database memory and avoid full-history fetches

Previous: Day 67 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–67, or equivalent SQL skills.

Goal

Connect SQL plan operations, concurrency and payload size to a bounded memory budget.

Before you start

Use at most two lab sessions and the existing two-day data. Keep changes transaction-local. Never change `postgresql.conf`, container memory or shared-server settings for this exercise.

Concept

`work_mem` is a budget for individual sort/hash operations, with additional details such as `hash_mem_multiplier` and parallel workers. It is not one memory allowance per database. Several operations and sessions can multiply demand. Four sessions with two four-MiB sorts suggest up to 32 MiB for those sort budgets alone; that is not a complete process-memory estimate. PostgreSQL's resource settings documentation describes these limits. Client arrays and JSON encoding consume separate memory.

Tiny example

```
SELECT name, setting, unit
FROM pg_catalog.pg_settings
WHERE name IN ('work_mem', 'hash_mem_multiplier',
               'max_parallel_workers_per_gather')
ORDER BY name;

BEGIN;
SET LOCAL work_mem = '1MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT point_id, count(*)
FROM student.ts_sample
WHERE site_id = 'SYN-NORTH'
  AND ts >= '2026-01-05 00:00Z'::timestamptz
  AND ts < '2026-01-06 00:00Z'::timestamptz
GROUP BY point_id;
ROLLBACK;
```

No spill is required on this small dataset. Report what the plan shows.

SQL challenge

Compare a server-side hourly kW aggregation with fetching one equipment's raw kW day and aggregating in an optional client. Return equal means and coverage. Measure rows transferred and encoded bytes; measure client memory only if you have a suitable tool. Write a budget for two concurrent requests that includes pool capacity, possible sort/hash operations and maximum response rows. Keep caching bounded by entries and bytes.

Experiment

Describe a cache key containing authorized site, point selection, range, resolution, query version and model/data revision. Change one key component and

show why reusing the old answer would be invalid. A missing revision policy is a limitation, not proof of freshness.

Acceptance checks

- No unbounded `SELECT *` is used as an analytics pipeline.
- Memory claims distinguish measured values from budget estimates.
- Aggregate eligibility and units match both paths.
- Local settings revert after the transaction.

Stretch and reflection

Which query operations could multiply memory beyond your first estimate?

Save your SQL, actual output, and one explanation in `student-work/sql/day68/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 67 ·

Day 69 — Measure bounded scale with declared limits

Previous: Day 68 · **Week 10 · 45–75 minutes.**
Prerequisites: Days 1–68, or equivalent SQL skills.

Goal

Produce a reproducible scale comparison whose data volume, concurrency and conclusions are explicit.

Before you start

Set a cap of 241,900 synthetic rows, two clients and twenty measured queries. Use a disposable scratch table, a five-second statement timeout and a total run deadline. Stop on repeated timeout, disk pressure or unexpected row growth.

Concept

Scale testing asks whether results and resource use remain acceptable as a declared dimension grows. It does not require millions of rows. Moving fixture copies forward in time tests a longer history while keeping point count constant; it does not simulate more buildings or ingestion contention. Report these limitations. Keep query parameters and indexes stable when comparing storage strategies, then explain any intentional difference.

Tiny example

```
SELECT count(*) AS proposed_rows,
       min(s.ts + g.n * INTERVAL '2 days') AS
         first_ts,
       max(s.ts + g.n * INTERVAL '2 days') AS
         last_ts
FROM course.sample AS s
CROSS JOIN generate_series(0, 9) AS g(n);
```

This computes the proposed shape without creating a larger table. It yields 241,900 rows and twenty days of shifted fixture data. Timestamps stay distinct within each original point.

SQL challenge

Choose three sizes up to the cap, such as one, three and ten fixture copies. Build your own scratch dataset and document its key, indexes and partition interval. Compare a fixed one-hour point query and a whole-test-range hourly aggregate. Record actual plans, row counts, database/extension versions, timings and query parameters. Compare raw aggregation with a deliberately refreshed summary where available. Validate answers before comparing speed.

Experiment

Run the six size/query pairs twice with one client: twelve statements. Choose two pairs for a two-client replay: four more statements. Count failed attempts toward the twenty-statement cap. Report individual timings; summarize distributions only when sample size supports them. Include timeout and failure counts. Do not describe a short local test as a production capacity guarantee.

Acceptance checks

- Generation stops at or below 241,900 rows, with no duplicate keys.
- Every query has an explicit time interval and result bound.
- Fixed-window answers remain equivalent as unrelated history grows.
- The report states which scale dimensions did and did not change.
- Scratch objects and any jobs have a documented cleanup plan.

Stretch and reflection

What next experiment would isolate point-count growth from history-length growth?

Save your SQL, actual output, and one explanation in `student-work/sql/day69/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 68 ·

Day 70 — Independent challenge — a Timescale performance report

Previous: Day 69 · **Week 10 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 64–69, or equivalent skills.

Project brief

Assess whether a Timescale design improves a declared historian workload. Use SQL, measured plans and a reproducible report. Your dataset may contain at most 241,900 synthetic rows; concurrency is capped at two clients and the run has explicit deadlines. Choose the design and comparisons independently.

Consult primary references and installed signatures as needed. Missing Timescale support is a blocked requirement, not a successful plain-PostgreSQL substitute.

Acceptance criteria

- Record PostgreSQL/extension versions, schema, unique key, chunk interval and indexes.
- Demonstrate selective time predicates and explain observed chunk pruning.
- Compare correct raw and continuous-aggregate answers, including a historical correction and freshness evidence.
- Measure a bounded columnstore comparison with matching result sets.

- Explain retention's effect on raw history and refreshed aggregates.
- Report actual timing, source/result rows, payload measurements where available, failures and memory estimates with their assumptions.

Deliverables

Submit SQL, a workload matrix, representative plans and a short recommendation tied to the workload. Include dataset generation limits and cleanup instructions. Label each result **observed**, **fixture-only**, **not run** or **blocked**. A tiny dataset showing no speedup is a valid finding.

Self-review

Which cost moved into refresh or ingest? What does the test fail to model? Which measured result would change your recommendation? Explain how a stale summary could return a fast but unacceptable answer.

Previous: Day 69 ·

Primary reading — Week 10

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 10

- Timescale hypertables
- Timescale generalized create_hypertable
- Continuous aggregate refresh
- Columnstore policy

- Pinned 2.30.2 policy SQL definitions
- PostgreSQL resource configuration